CLR

Common Language Runtime

CLR (Common Language Runtime) nədir?

- **CLR**, yəni **Common Language Runtime**, .NET platformasının **əsas yürütmə mühərrikidir**. O, .NET kodunu işlədən, idarə edən və qoruyan bir “virtual maşındır”.

Compile (kompilyasiya) nədir?

- **Compile etmək** — proqramlaşdırma dilində (məsələn, C#) yazılmış **mənbə kodunu** (source code) **başqa bir formata çevirib işlədilə biləcək hala gətirməkdir.**

Addım	Nə baş verir?
1. Mənbə kod yazılır	Məsələn: Program.cs faylında Console.WriteLine("Salam"); yazırsan.
2. Compile olunur	Bu kod **C# Compiler** tərəfindən oxunur və **IL (Intermediate Language)** koduna çevrilir. Bu, .exe və ya .dll faylında saxlanılır.
3. CLR və JIT	CLR bu IL kodu götürür və **JIT compiler** vasitəsilə **maşın koduna** çevirib prosessora ötürür.

#Jit – (just in time) proqram işə düşdükdə avtomatik olaraq JIT kompilyatoru işləyərək kodları maşın dilinə çevirir

Compile nə üçündür?

- Kod **prosessorun başa düşdüyü dildə** deyil, insan üçün yazılıb (C#).
- Kompilyator onu **ara kod** (IL) şəklinə çevirir ki, CLR onu idarə etsin.
- JIT isə son mərhələdə bu ara kodu **maşın koduna** çevirir — yəni prosessorun başa düşdüyü 1 və 0-lara.

Console.WriteLine("Salam Dünya!");

- C# dilində yazılır
- Compile olunduqdan sonra .exe faylı yaranır
- .exe faylı işə salındıqda CLR kodu JIT vasitəsilə maşın koduna çevirib icra edir
- Ekranda “Salam Dünya!” çıxır

CLR-in Əsas Vəzifələri:

Vəzifə	İzah
Yaddaş İdarəetməsi	Obyektlərin yaradılması və silinməsi ilə məşğul olur. Yaddaş ayırma və Garbage Collector (GC) onun nəzarətindədir.
Code Execution (İcra)	C# və ya digər .NET dillərində yazılmış kodu maşın dilinə çevirir və işlədir.
Type Safety	Tip təhlükəsizliyini təmin edir. Yanlış tip çevrilmələrinin qarşısını alır.
Exception Handling	İstisnaları (xətaları) idarə edir, tətbiqin çökməməsi üçün sistemli cavab verir.
Security	Kodun təhlükəsizliyini təmin edir (Code Access Security, Role-based security və s.)
Thread Management	Paralel prosesləri (multi-threading) idarə edir.
Interop Support	Unmanaged kodlarla (C/C++ DLL) işləməyə imkan verir.

Managed Code nədir?

- **Managed code** – CLR-in nəzarəti altında işləyən koddur.
- CLR bu kodun yaddaşını,təhlükəsizliyini,tip uyğunluğunu,exception-larını idarə edir.
- Kod əvvəlcə C# IL çevrilir.
- CLR bunu JIT Compiler ilə maşın koduna çevirir və icra edir.

```
public class Adam {
    public string Ad { get; set; }
}

Adam valeh = new Adam(); // CLR yaddaş ayırır
```

- new Adam() zamanı heap də obyekt yaranır.
- Bu obyekt üçün CLR yaddaş ayırır.
- Garbage Collector bu obyektə heç bir referans qalmadıqda onu təmizləyəcək.

Garbage Collector (GC) necə işləyir?

- GC CLR ın bir hissəsidir və Managed Heap üzərində işləyir
- **GC mərhələləri:**

1. **Mark** – İstifadədə olan obyektləri tapır (referans verilmiş).
2. **Sweep** – Referansı olmayan obyektləri "zibil" kimi tanıyır.
3. **Compact** – Heap-də boşluqları birləşdirir, daha sıx yaddaş verir.

GC necə çağrılır?

- Avtomatik, CLR tərəfindən (sən çağırmırsan)
- İstəsən, GC.Collect() ilə **məcburi** çağıra bilərsən (amma bu tövsiyə olunmur).

CLR ilə GC arasındakı əlaqə

•CLR, IL kodunu icra edən mühərrikdir.
•CLR **GC**-ni öz daxilində işə salır.
•GC isə **Managed Heap** üzərində obyektləri təmizləyir.

•string name = "Valeh";
•Burada:
•name dəyişəni heap-də yaranır.
•CLR bu yaddaşı ayırır.
•GC isə sonra onu təmizləyir(əgə istifadə edirmirsə)

- C# kod → IL (Intermediate Language) → CLR
- ↓
- JIT Compiler
- ↓
- Native (maşın) kod
- ↓
- CLR Memory Allocation (Heap)
- ↓
- GC (zibil obyektləri təmizləyir)

Unmanaged Code

- Unmanaged Code — .NET-in idarəetməsi **xaricində işləyən** koddur. Bu, **CLR (Common Language Runtime)** tərəfindən avtomatik idarə olunmur, yəni:
- **Garbage Collector (GC)** tərəfindən yaddaş təmizlənmir
- **Exception handling** (istisnaların idarəsi) .NET-in mexanizmləri ilə olmur
- Yaddaşın ayrılması və sərbəst buraxılması proqramçının **öz məsuliyyətindədir**

Xüsusiyyətlər	Managed Code	Unmanaged Code
İcra mühiti	CLR	Native OS
Yaddaş idarəsi	GC ilə	Proqramçı tərəfindən
Təhlükəsizlik	Yüksək	Aşağı
Performans	Mülayim	Daha yüksək ola bilər
Məsuliyyət	CLR	Proqramçı

- https://medium.com/kodera/common-language-runtime-nec%C9%99-i%CC%87%C5%9Fl%C9%99yir-849e4b793dd3